Module - 7:
Testing Foundation

- Amjad Hossain
18th Feb, 2026

Test Ownership Model

Testing is shared responsibility

Developers ● Unit tests ● Component tests ● Integration tests ● Contract tests	src/main/java/... src/test/java/ → Unit tests src/component-test/java/ → Component tests src/integration-test/java/ → Integration tests src/contract-test/java/ → Contract tests
Platform/DevOps ● CI enforcement ● Test coverage gates ● Environment stability	**QA** ● Exploratory/Manual ● E2E ● Business validation

Developer: Unit Tests

Purpose

Test **business logic in isolation**.

- No Spring context
- No DB
- No HTTP
- No Kafka
- No external systems

Target layers:

- `domain`
- `application.service`

src/test/java/com/bank/platform/customeraccount

```
├── domain
│   └── AccountTest.java
│
└── application
    └── service
        └── AccountOpeningServiceTest.java
```

Coverage Targets

- 90-95% line coverage
- 80%+ branch(decision) coverage

Developer: Component Tests

Purpose

Test a **single service/Modules internally**.

- Other service
- Real message queue
- External REST systems

Target layers:

- `Controller`
- `Service`
- `Repository`
- `Transaction`
- `Validation`
- `Security`

src/component-test/java/com/bank/platform/customeraccount

```
├── AccountOpeningComponentTest.java
├── AccountClosureComponentTest.java
└── config
    └── TestContainerConfig.java
```

Coverage Focus

- 100% critical APIs
- 100% transaction scenarios
- 60–75% overall service behavior

Developer: Integration Tests

Purpose

Test collaboration between

- Multiple services
- DB + Kafka
- Service A → Service B
- Module A → Module B
- Event publishing & consumption

Focus:

- `Cross-service correctness`

src/integration-test/java/com/bank/platform/customeraccount

├── AccountToPaymentFlow**IT**.java
└── CustomerEventPropagation**IT**.java

Developer: Contract Tests

Purpose

Prevent breaking API changes between

- Services
- Modules
- 3rd Party Service

Focus:

- `Request structure`
- `Response schema`
- `Status codes`
- `Error models`
- `Required fields`

```
src/contract-test/java/com/bank/platform/customeraccount
 |
 ├── PaymentServiceContractTest.java
 └── PaymentProviderVerificationTest.java
```

Coverage Target

- 100% public APIs
- 100% error responses

QA: Exploratory/Manual Testing

Purpose

Discover unknown defects through **human-driven investigation**

- Not scripted
- Not automated
- Find what automation cannot anticipate

Focus:

- `UI behavior inconsistencies`
- `Edge cases not captured in test cases`
- `Unexpected combinations`
- `Usability flaws`
- `Error message clarity`
- `Security loopholes`
- `Data anomalies`

Enterprise structure:

```
test-management/
├── exploratory-charters/
│   ├── AccountOpening_Charter.docx
│   ├── HighValueTransaction_Charter.docx
│   │
└── findings/
    └── Q3Sprint4_Findings.xlsx
```

Stored in:

- Test management tool (Jira/Xray/TestRail)
- Confluence charters

QA: Exploratory/Manual Testing

Example Exploratory Charter

Title: High-Value Withdrawal Scenario **Explore:** • Withdrawals > 100,000 • Rapid consecutive withdrawals • Different currency combinations • Partial network interruptions **Expected:** • Proper limit validation • Clear rejection message • No partial transaction	**Tools** ● Jira / Xray ● TestRail ● Browser DevTools ● Postman ● SQL clients (for DB inspection)

QA: E2E (End-To-End) Testing

Purpose

- Validate complete business journey across systems
- Tests real system flow
- Only critical business flows

Focus:

- UI
- API Gateway
- Multiple services
- Database
- Messaging
- Security

Enterprise structure:

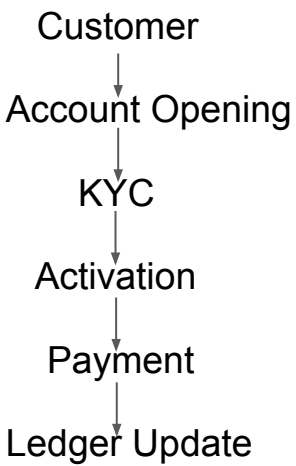
```
e2e-tests/
  ├── src/test/java/com/bank/e2e
  │      ├── AccountOpeningFlowE2ETest.java
  │      ├──
LoanDisbursementFlowE2ETest.java
  └── test-data/
        └── high_value_accounts.json
```

Stored in:

- Usually separate repository

QA: E2E (End-To-End) Testing

Example



Tools

- Selenium / Playwright
- Cypress
- RestAssured
- Cucumber

QA: Business Validation Testing

Purpose

Ensure system behavior matches **business rules and regulatory requirements**.

- Financial accuracy
- Regulatory constraints
- Reporting correctness
- Audit trail completeness

Focus:

- `Compliance`
- `Regulatory calculations`
- `Maker-checker rules`
- `Interest calculation correctness`
- `Accounting reconciliation`

Enterprise structure:

```
business-validation/
 ├── BRD-Test-Mapping.xlsx
    ├── Regulatory-Validation-Suite.docx
```

Automation layer may exist under:

src/business-validation-test/java/...

QA: Business Validation Testing

Example

Title: High-Value Withdrawal Scenario **Input:** • Principal: 1,000,000 • Rate: 5% • Period: 12 months **Expected:** • Value matches regulatory formula • Rounding policy applied correctly	**Tools** ● Excel-based validation models ● SQL reconciliation scripts ● Custom validation frameworks ● Cucumber for rule validation

Coverage Strategy:

- 100% coverage of BRD rules
- 100% regulatory compliance clauses
- Cross-verification with finance models

Security Testing Overview

Security is not a QA-only responsibility

1. Static Application Security Testing (SAST)
2. Dynamic Application Security Testing (DAST)
3. Dependency / Vulnerability Scanning
4. API Security Testing
5. Penetration Testing
6. Authentication / Authorization validation

Static Security Testing (SAST)

Purpose

Analyze source code for vulnerabilities before runtime.

Detect

- SQL injection
- Hardcoded credentials
- Unsafe deserialization
- Path traversal
- Weak cryptography usage

Runs

- On every pull request

Tools

- GitHub/Azure/AWS Advanced Security
- SonarQube (Security rules)
- Checkmarx
- Fortify
- Semgrep

Responsibility

Primary: Developers

Governance: Security Team

Enforcement: Platform (CI pipeline gates)

Dynamic Security Testing (DAST)

Purpose

Test running application for runtime vulnerabilities.

Detect

- Injection attacks
- XSS
- Broken authentication
- Misconfigured headers
- CSRF

Runs

- Before major release
- In staging

Tools

- OWASP ZAP
- Burp Suite
- Netsparker
- AppScan

Responsibility

Primary: Security + QA
Support: Developers
Platform: Provide staging environment

API Security Testing

Purpose

- Functional API correctness
- Security validation

Focus

- Authorization checks
- Role-based access
- Token expiration
- Rate limiting
- Payload tampering
- Missing field injection
- IDOR (Insecure Direct Object Reference)

Runs

- Before major release
- In staging
- With limited scope in Prod

Tools

- Postman (Security test cases)
- RestAssured
- OWASP ZAP API mode
- Burp Suite
- Karate DSL

Responsibility

Functional API tests: Dev + QA
Security API tests: Security + QA
Access control logic: Developers

Load & Performance Testing

Purpose

Validate system behavior under expected and peak load

- Load testing (expected traffic)
- Stress testing (beyond limits)
- Spike testing (sudden bursts)
- Endurance testing (long-running)
- Soak testing (Continuous huge load & monitor)

Focus

- Authorization checks
- Role-based access
- Token expiration
- Rate limiting
- Payload tampering
- Missing field injection
- IDOR (Insecure Direct Object Reference)

Runs

- In production environment

Tools

Load

- JMeter
- Gatling
- K6
- Locust
- BlazeMeter

Monitoring

- Prometheus
- Grafana
- ELK
- APM tools (New Relic, Dynatrace, AppDynamics)

Load Testing Responsibility

Architectect/Lead:

- Define TPS targets
- Define p95/p99 latency

Developers:

- Write performance-aware code
- Optimize queries
- Memory leaks
- Optimize I/O

QA:

- Design realistic scenarios

Platform:

- Provide production-like infra

Security:

- Validate rate limiting

Penetration Testing (PenTest)

Purpose

Simulate **real-world attacker behavior** to discover exploitable vulnerabilities

- Manual + automated
- Offensive security mindset
- Tests system as an attacker would

Focus:

- `Authentication bypass`
- `Privilege escalation`
- `Broken access control`
- `IDOR (Insecure Direct Object Reference)`
- `SQL Injection (manual exploitation)`
- `Business logic abuse`
- `Rate limit bypass`
- `JWT manipulation`
- `Session hijacking`
- `Misconfigured cloud services`

Enterprise structure:

```
security/
 ├── pentest-reports/
 │     ├── 2026_Q1_Internal_Pentest.pdf
 │     ├── 2026_Q1_External_Pentest.pdf
 ├── remediation-tracker.xlsx
```

Stored in:

- GRC (**G**overnance, **R**isk, and **C**ompliance) tools
- Security vault
- Restricted access storage

Penetration Testing (PenTest)

Tools

Automated tools:

- Burp Suite Pro
- OWASP ZAP
- Metasploit
- Nikto
- Nmap

Manual techniques:

- Token replay
- Parameter tampering
- Privilege escalation attempts

Cloud:

- ScoutSuite
- Prowler

Responsibility

Primary: Security Team (or external certified vendor)

Support:

- Dev → Fix vulnerabilities
- Platform → Fix infra misconfigurations
- Architecture → Approve design changes

Frequency:

- Before major releases
- Annually (minimum)
- After major architectural change

Penetration Testing Coverage Strategy

Not coverage driven but …

Measured by:

- OWASP Top 10 coverage
- Critical vulnerability count
- Exploit success rate
- Remediation SLA

Severity Levels:

- **Critical → Immediate block release**
- **High → Fix before release**
- **Medium → Planned**
- **Low → Risk accepted / backlog**
- **Safe → No Issue**

Authentication Validation

Purpose

Ensure identity mechanisms cannot be bypassed

Covers:

- Login flow
- Token generation (JWT/OAuth2)
- Token expiration
- Refresh token misuse
- Brute-force protection
- MFA enforcement
- Password policy enforcement

Runs:

- Automated in CI (basic checks)
- Deep validation in security cycle

Tools

- Postman security tests
- RestAssured
- OWASP ZAP
- Burp Suite
- Custom JWT tampering scripts

Responsibility

Implementation: Developers
Test scenarios: QA + Security
Policy definition: Security + Architect/Lead

Authorization Validation

Purpose

Ensure role-based and permission-based access control works correctly

Covers:

- Role-based access (ADMIN, MAKER, CHECKER)
- Horizontal access control (user cannot access another user's data)
- Vertical privilege escalation (user cannot call admin API)
- Field-level masking
- API-level permission checks

Runs:

- Automated in CI (basic checks)
- Deep validation in security cycle

Tools

- Spring Security test utilities
- RestAssured
- Postman role switching
- Automated role-matrix validation scripts

Responsibility

Code-level enforcement: Developers
Test cases: QA
Policy & compliance: Security
CI enforcement: Platform

Authorization Validation (**Example**)

User A calls:

`GET /accounts/{accountId}`

Test:

- If accountId belongs to User B → **403**
- If user role insufficient → **403**
- If token missing → **401**

```
@Test
void shouldRejectAccessForUnauthorizedRole()
{

    given()
        .auth().oauth2(userToken)
    .when()
        .delete("/admin/accounts/123")
    .then()
        .statusCode(403);
}
```

Thank you